



CS100 Recitation 13

Makefile

@lichf2025@shanghaitech.edu.cn
@heyx2025@shanghaitech.edu.cn

Contents

- 1 Introduction & Installation ->
- 2 0. The Hard Way ->
- 3 1. Basic Rules ->
- 4 2. Variables ->
- 5 3. Header Dependencies ->
- 6 4. Phony Targets ->
- 7 5. Advanced Features ->

Introduction & Installation

What is Make?

- **Make**: 一个自动化构建工具，通过读取 Makefile 来决定如何编译程序。
- **Makefile**: Make 的配置文件（默认名称 `Makefile`），描述了文件之间的依赖关系。

Makefile 最初为 C/C++ 设计，但适用于任何基于文件依赖的构建系统（如 LaTeX, Java 等）。

如果说手动编译文件是项目构建中的“手敲二进制”，Makefile 就是构建中的汇编代码。后续的 CMake、XMake 等工具，就如同 C /C++ 等更加高级的语言。

Installing Make

Via Package Manager (Ubuntu)

通常包含在 `build-essential` 中。

```
sudo apt install make
```

Compiling from Source (源码编译)

为什么? Make 4.4+ 对 `.NOTPARALLEL` 支持更好。Ubuntu 24.04 默认是 4.3。

```
wget https://ftp.gnu.org/gnu/make/make-4.4.1.tar.gz # 1. Download
tar -xzf make-4.4.1.tar.gz                        # 2. Extract
./configure --prefix=/usr/local                    # 3. Configure
make && make install                                # 4. Build & Install
```

0. The Hard Way

场景假设

假设我们有一个简单的 C++ 项目：

- `main.cpp`: 主程序
- `utils.cpp`: 工具函数实现
- `utils.h`: 工具函数声明

手动编译

```
g++ main.cpp utils.cpp -o app
```

问题

1. **繁琐**: 每次都要敲长命令。(可以通过写成脚本来解决)
2. **低效**: 即使只改了 `main.cpp`, `utils.cpp` 也会被重新编译 (全量编译)。

1. Basic Rules

Makefile 的核心：规则 (Rules)

Target (目标): Dependencies (依赖)
Command (命令) <-- 必须以 TAB 开头

我们的第一个 Makefile

```
all: main.cpp utils.cpp
    g++ main.cpp utils.cpp -o app
```

相当于把手动编译命令一一对应的写进了 Makefile。

Make 的智慧：增量编译

我们首先将上面的 Makefile 改写成更合理的形式：

```
app: main.o utils.o
    g++ main.o utils.o -o app

main.o: main.cpp utils.h
    g++ -c main.cpp -o main.o

utils.o: utils.cpp utils.h
    g++ -c utils.cpp -o utils.o
```

在这个 Makefile 中，我们将大的编译过程拆分成了更小的 Targets, 每个 Target 只负责生成一个文件。

Make 的智慧：增量编译

Make 是“懒惰”的（也是聪明的）：

1. **Target 不存在**：执行命令。
2. **Target 存在**：检查 **时间戳 (Timestamp)**。
 - 如果任何一个 **Dependency** 比 **Target** 新 -> **重新编译**。
 - 如果 **Target** 比所有 **Dependency** 都新 -> **跳过**。

这样，如果我们只修改 `main.cpp`，使用刚才的新 Makefile，则 Make 只会重编 `main.o` 和 `app`，而不会重编 `utils.o`。

2. Variables

消除硬编码 (Hardcoding)

上面的 Makefile 把编译器 `g++` 写死了。如果想换 `clang++` 怎么办?

使用变量

```
CXX := g++
CXXFLAGS := -Wall -g
app: main.o utils.o
    $(CXX) main.o utils.o -o app
main.o: main.cpp utils.h
    $(CXX) $(CXXFLAGS) -c main.cpp -o main.o
# ...
```

- 定义: `VAR := value`
- 引用: `$(VAR)` 或 `${VAR}`

变量赋值方式

- **:= (Simple/Immediate)** (推荐默认使用):
 - **立即展开**。定义时就确定值。类似 C++ 的“传值”。
- **= (Recursive)**:
 - **延后展开**。使用时才确定值。类似 C++ 的“传引用”。

```
X = foo
Y = $(X) bar    # Y 存储的是表达式 "$ (X) bar", 不是计算结果
X = baz         # 修改 X
all:
    @echo "X = $(X)" # 输出: baz
    @echo "Y = $(Y)" # 输出: baz bar (使用时才展开, 所以用新的 X)
```

- **?= (Conditional)**:
 - 如果变量**未定义**, 则赋值。常用于允许用户通过命令行覆盖配置。
 - 非常适合作为参数的默认值设置!

自动化变量与模式规则

每个 `.cpp` 都要写一遍规则太麻烦了。

Pattern Rule (模式规则)

使用 `%` 通配符：

```
%.o: %.cpp  
    $(CXX) $(CXXFLAGS) -c $< -o $@
```

Automatic Variables (自动化变量)

- `$@`: **Target** (目标文件)
- `$<`: **First Dependency** (第一个依赖)
- `$$`: **All Dependencies** (所有依赖)

自动扫描源文件

如果文件很多，手动列出 `main.o utils.o ...` 也很累。

常用函数

- **wildcard**: 扫描匹配的文件。

```
SRC := $(wildcard *.cpp) # 结果: main.cpp utils.cpp
```

- **patsubst**: 模式替换。

```
OBJ := $(patsubst %.cpp, %.o, $(SRC)) # 结果: main.o utils.o
```

The `shell` Function (Shell 函数)

`wildcard` 功能有限，有时我们需要更强大的命令（如递归查找、系统检测）。这时候，我们可以使用 `shell` 函数调用系统命令：

```
# Platform
UNAME := $(shell uname -s)

# Source directory
SRC_DIR := src
SRCS := $(shell find ${SRC_DIR} -name "*.cpp")
```

Makefile 会执行 `uname -s` 和 `find ...`，并将输出结果赋值给变量。

3. Header Dependencies

The Problem: Header Files

场景: 你修改了 `utils.h`, 但没有修改 `main.cpp`。

结果: Make 认为 `main.o` 是最新的, **不会重新编译**。

原因: Makefile 只知道 `main.o: main.cpp`, 不知道它还依赖 `utils.h`。

我们需要让 Make **自动学会**这些依赖关系!

Solution: Automatic Dependency Generation

原理:

1. **编译期**: 编译器 (`gcc/g++`) 分析 `#include`, 生成依赖文件 (`.d`)。
2. **读取期**: Makefile 使用 `-include` 指令读取这些 `.d` 文件。

关键 Flags:

- `-MMD`: 生成依赖文件 (忽略系统头文件)。
- `-MP`: 为头文件生成伪目标 (防止删除头文件后报错)。

Implementation

```
DEPFLAGS := -MMD -MP
```

```
# 1. 在编译命令中加入 DEPFLAGS
```

```
%.o: %.cpp
```

```
    $(CXX) $(CXXFLAGS) $(DEPFLAGS) -c $< -o $@
```

```
# 2. 引入生成的依赖文件 (.d)
```

```
# 使用 "-" 忽略文件不存在的错误 (首次编译时)
```

```
DEPS := $(OBJ:.o=.d)
```

```
-include $(DEPS)
```

How it works

编译 `main.cpp` 时, 生成 `main.d`:

```
main.o: main.cpp utils.h  
utils.h:
```

Makefile 执行 `-include main.d` 后, 内存中增加了上面的规则。
现在, **修改 `utils.h` 会触发 `main.o` 重编!**

4. Phony Targets

.PHONY (伪目标)

有些目标不是为了生成文件，而是执行动作（如清理、运行）。

```
.PHONY: clean run
```

```
clean:
```

```
    rm -f app *.o
```

```
run: app
```

```
    ./app
```

- 为什么要用 **.PHONY**?

- 如果恰好有一个文件叫 `clean`，`make clean` 会以为目标已完成而不执行。
- 声明为 **.PHONY** 强制 Make 忽略文件检查，总是执行命令。
- 类似的，有一些仿真操作依赖外部文件的修改，或者需要反复重新运行而不修改内容，也可以声明为 **.PHONY**。

5. Advanced Features

宏 (Macros)

定义多行命令块，封装复杂逻辑。

```
define NOTIFY_DONE
    @echo "-----"
    @echo "Build [$@] Completed!"
    @echo "-----"
endif

app: $(OBJ)
    $(CXX) $(OBJ) -o app
    $(NOTIFY_DONE)
```

条件判断 (Conditionals)

根据变量值改变构建行为（如 Debug/Release 模式）。

```
DEBUG ?= 1
ifeq ($(DEBUG), 1)
    CXXFLAGS += -g -00
else
    CXXFLAGS += -O2
endif
```

使用: `make DEBUG=0` (构建 Release 版)

命令前缀 (Command Prefixes)

控制命令的回显和错误处理。

- **@: 静默执行**。只输出命令结果，不打印命令本身。
- **-: 忽略错误**。即使命令失败（如删除不存在的文件），也继续执行。

clean:

```
-rm *.o          # 即使文件不存在也不报错  
@echo "Clean Done!" # 只输出 "Clean Done!"
```

目标特定变量 (Target-specific Variables)

只为某个特定目标修改变量值。

```
# 默认是 Release  
CXXFLAGS := -O2  
  
# 当执行 make debug 时, CXXFLAGS 会追加 -g  
debug: CXXFLAGS += -g  
debug: app
```

这比全局修改变量更灵活!

Summary

Summary

我们的 CS100 HW 7 将使用 Makefile 来管理编译流程，希望这节课的内容可以帮助大家更好的理解和使用 Makefile~

这学期的 CS100 Tutorial 就到这里啦，祝大家学习顺利，项目成功！